

CSCI11 Final Exam REVIEW — Answer Key

Q	Ans	Reason
1	c	Alan Turing developed the theoretical model of a universal computing device
2	c	Moore's Law: transistor count doubles approximately every two years
3	d	3-bit unsigned: values 0 through $2^3 - 1 = 7$
4	d	$2^{10} = 1024$
5	b	Two's complement negation: flip every bit, then add 1
6	d	-3 in 4 bits: start with 3 = 0011, NOT = 1100, +1 = 1101
7	c	Overflow: both operands have the same sign AND the result has the opposite sign
8	b	AND with 0 clears a bit; used to mask/clear selected bits
9	c	XOR of two identical bits = 0 (they "cancel out")
10	c	Binary 1100 = 8+4 = 12 decimal = C hex
11	d	De Morgan's: NOT(A OR B) = (NOT A) AND (NOT B)
12	c	All Turing-complete computers can compute the same set of problems given enough time and memory
13	b	A multiplexer selects one of its inputs to pass to the output — it is a data selector
14	d	An n-input decoder activates exactly one of 2^n output lines
15	d	Increasing transistor density raises power consumption and generates heat
16	d	JSR saves PC+1 (address of next instruction) into R7, then jumps to the subroutine
17	b	LDR uses base register + 6-bit offset (base-plus-offset addressing)
18	c	Data before code: must start with an unconditional BR to jump over the data to the first instruction
19	c	PSR[15] = 1 → Supervisor (privileged) mode
20	b	LC-3 null terminator = x0000 (all 16 bits zero)
21	d	RTI pops both PC and PSR from the supervisor stack, restoring privilege level and condition codes
22	c	Before an inner JSR, the outer subroutine must save R7 (the outer caller's return address)
23	d	Trap vector table occupies x0000–x00FF
24	d	STI R0, PTR: load address from PTR (x5000), then store R0 to x5000
25	a	LD uses PC-relative addressing: effective address = PC + sign-extended 9-bit offset
26	c	R5 is the frame pointer — it stays fixed during a function call so locals have stable offsets
27	c	Return value slot is always at R5+3 (above saved R5, saved R7)
28	b	Right-to-left: last (rightmost) argument pushed first so first argument ends up on top
29	b	<code>void</code> means the function returns no value
30	d	Pass by value: function receives a copy; changes inside the function do not affect the caller
31	d	With right-to-left push convention, first (left) parameter lands at R5 + 4
32	d	SIMD = Single Instruction, Multiple Data
33	b	<code>%c</code> formats a value as an ASCII character
34	d	A system call requires a CPU mode switch from user mode to kernel mode
35	b	<code>xor eax, eax</code> is 2 bytes vs <code>mov eax, 0</code> (5 bytes); modern CPUs recognize it as a zeroing idiom

Debug 1 — Missing add back to remainder

Bug: After determining the number of times *second* goes into *first*, the remainder needs to be calculated. To do so, add *second* back to R1, to determine what is left over. **Fix:** Load and Add back to R2:

```

                ; divide
                .orig x3000
first           .fill #105      ; first / second
second          .fill #10
                br start
                LD  R2, second ;
                LD  R1, first ;

                ; result in R0
                ; remainder in R1
                AND R0, R0, #0
                NOT R2, R2
                ADD R2, R2, #1
subtr           ADD R1, R1, R2
                BRn remain
                ADD R0, R0, #1
                BR subtr
remain          LD  R2, second
                ADD R1, R2, R1

                HALT
                .end

```

Debug 2 — Inner JSR clobbers R7 (infinite loop)

Bug: `GREET` is entered via JSR, which places the caller's return address in R7. Inside `GREET`, `JSR NEWLINE` overwrites R7 with the address of the instruction immediately following it — which is the `RET` at the end of `GREET`. When that `RET` executes, it jumps to R7, landing back on itself. The result is an infinite loop: `RET` → `RET` → `RET` ...

Fix: Save R7 before the inner `JSR` and restore it afterward:

```

GREET          ST   R7, SAVE_R7 ; save caller's return address
                LEA  R0, HI_MSG
                PUTS
                JSR  NEWLINE
                LD   R7, SAVE_R7 ; restore caller's return address
                RET

NEWLINE        LD   R0, NL_CHAR
                OUT
                RET

HI_MSG         .STRINGZ "Hi!"
NL_CHAR        .FILL x000A
SAVE_R7        .BLKW 1

```

Debug 3 — Trap handler uses HALT instead of RTI

Bug: The trap handler ends with `HALT` (= `TRAP x25`), which terminates the program completely. A trap handler must end with `RTI` to pop the saved PC and PSR from the supervisor stack and return control to the user program that called the trap.

Fix: Replace `HALT` with `RTI` :

```
.ORIG x0300
CLEAR_TRAP AND R0, R0, #0      ; R0 = 0
           RTI                  ; ← was HALT; RTI returns to user program
           .END
```

Debug 4 — Wrong branch condition (BRn instead of BRp)

Bug: After `ADD R1, R1, #-1` (decrement count), the condition codes reflect the new value of R1. Starting with R1 = 8:

- After first iteration: R1 = 7 (positive) → `BRn` is **not** taken (7 is not negative) → loop exits after one element.

`BRn` means "branch if the result is negative," but the loop should continue while the counter is still positive (greater than zero).

Fix: Change `BRn` to `BRp` :

```
ADD R1, R1, #-1      ; decrement count
BRp LOOP             ; ← was BRn; continue while count > 0
```

Binary Math — Answers with Work

All numbers are 8-bit two's complement.

1. 00110011 + 00011001

```
00110011 = +51
+ 00011001 = +25
-----
01001100 = +76
```

Both operands positive, result positive → **no overflow**.

Answer: **01001100**

2. 10100011 + 11011100

```
10100011 = -93
+ 11011100 = -36
-----
1 01111111 (carry out; result = 01111111 = +127 as signed)
```

Both operands negative (MSB = 1), result positive (MSB = 0) → **overflow**.

($-93 + -36 = -129$, which is below the 8-bit minimum of -128 .)

Answer: **overflow**

3. 11010101 - 01001010

Convert subtrahend to two's complement: NOT 01001010 = 10110101, +1 = 10110110

```

11010101 = -43
+ 10110110 = -74 (negated subtrahend)
-----
1 10001011 (carry out; result = 10001011)

```

Both operands negative; result 10001011 = -117 (negative) → **no overflow**.

Answer: **10001011**

4. 01001010 + 01000110

```

01001010 = +74
+ 01000110 = +70
-----
10010000 = -112 (as signed)

```

Both operands positive (MSB = 0), result negative (MSB = 1) → **overflow**.
(+74 + +70 = +144 > 127, the 8-bit maximum.)

Answer: **overflow**

5. 10111100 + 01011010

```

10111100 = -68
+ 01011010 = +90
-----
1 00010110 (carry out; result = 00010110 = +22)

```

Operands have opposite signs → **no overflow possible**.

Answer: **00010110**

Boolean Algebra Answer Key

Q	Ans	Identity / Law
1	d	A OR A = A (OR Idempotent)
2	c	B AND 1 = B (AND Identity)
3	a	NOT(A OR B) = (NOT A) AND (NOT B) (De Morgan's — OR form)
4	d	A OR (A AND B) = A (Absorption Law)
5	b	(A AND B) AND 0 = 0 (AND Null/Dominance element)
6	d	A AND 0 = 0 (AND Null element)
7	c	A XOR 0 = A (XOR identity element)
8	d	NOT(NOT A) = A (Double Negation / Involution)
9	c	A AND NOT A = 0 (Complement Law — AND)
10	d	A OR NOT A = 1 (Complement Law — OR)

Markdown — Model Answer

```
# Course Notes

## Introduction

## References

1. [Patt and Patel](textbook.htm)
2. [LC-3 Simulator](lc3tools.pdf)

## Diagrams

- ![Circuit 1](circuit1.png)
- ![Circuit 2](circuit2.png)
```

Key syntax points:

- `#` = heading 1; `##` = heading 2 — one blank line before and after each heading
- `[link text](url)` — hyperlink; both a web URL and a local filename use the same syntax
- `![alt text](filename)` — image; the `!` before `[` is required
- Numbered list: `1.` then space then text; bulleted list: `-` or `*` then space then text
- `Introduction` is a heading 2 with no content below it — just the `##` line is sufficient

Digital Design — Answers

(Questions use the same images as Test 1)

1. AND gate — What is the output?

Output: 0

The AND gate requires ALL inputs to be 1 to output 1. With one input at 0, the output is forced to 0 regardless of the other input.

2. OR gate — What is the output when the button is pressed?

Output: 1

Pressing the button connects a logic 1 to the OR gate input. OR outputs 1 when at least one input is 1, so the output becomes 1.

3. XOR gate — What is the output when the button is pressed?

Output: 0

With the button pressed, both inputs to the XOR gate are 1. XOR outputs 1 only when the inputs differ; identical inputs (both 1) produce 0.

4. NOT gate — What is the output?

Output: 1

The NOT gate's input is 0 (switch open / no connection). NOT inverts its input: NOT 0 = 1.

5. D flip-flop — If D = 1, what is Q?

Q = 1 (after the active clock edge)

A D flip-flop captures the value of D on the rising clock edge and presents it at output Q. With D = 1, Q becomes 1 on the next clock edge.

Program LC-3 Assembly — Model Solutions

Problem 1 — Count Uppercase Letters in a String

```

; Assignment: Final Review Problem 1
; Name:      [Student Name]
; Description: Traverses TESTSTR and counts uppercase letters (x41–x5A).
;            Stores the count in COUNT, then halts.
; Registers:
;   R0 – current character loaded from string
;   R1 – pointer to current character (advances each iteration)
;   R2 – running count of uppercase letters found
;   R3 – scratch register for range-check arithmetic

        .ORIG x3000

        LEA R1, TESTSTR    ; R1 = address of first character
        AND R2, R2, #0     ; R2 = count = 0

LOOP    LDR R0, R1, #0      ; R0 = current character
        BRz DONE           ; null terminator → finished

        ; Check R0 >= 'A' (x41 = 65): R0 - 65 must be ≥ 0
        LD  R3, NEG_A       ; R3 = -65
        ADD R3, R0, R3      ; R3 = R0 - 65
        BRn NEXT           ; R0 < 'A' → not uppercase

        ; Check R0 <= 'Z' (x5A = 90): R0 - 90 must be ≤ 0
        LD  R3, NEG_Z       ; R3 = -90
        ADD R3, R0, R3      ; R3 = R0 - 90
        BRp NEXT           ; R0 > 'Z' → not uppercase

        ; Character is uppercase – increment count
        ADD R2, R2, #1

NEXT    ADD R1, R1, #1      ; advance pointer to next character
        BR  LOOP

DONE    ST  R2, COUNT       ; store final count
        HALT

NEG_A   .FILL xFFBF        ; -65 (two's complement of 'A' = x41)
NEG_Z   .FILL xFFA6        ; -90 (two's complement of 'Z' = x5A)

TESTSTR .STRINGZ "Hello World"
COUNT  .BLKW 1

        .END

```

Problem 2 — MAX2 Subroutine with Full Calling Convention

```

; Trap x26 or TRAP_MAX
; Find the maximum value in an array
; on entry:
; R0 - array address
; R1 - size of array
; on return
; R0 - maximum value

; section one - trap vector
    .ORIG x0026
    .fill TRAP_MAX ; x0026
    .END

; section two - trap code
    .ORIG x300

; save registers used, R2, R3, R4
TRAP_MAX
    ADD R6, R6, #-1
    STR R2, R6, #0
    ADD R6, R6, #-1
    STR R3, R6, #0
    ADD R6, R6, #-1
    STR R4, R6, #0

; initialize R3 to first element as initial max
; go to next element and decr count
    LDR R3, R0, #0
    ADD R0, R0, #1
    ADD R1, R1, #-1

; loop through array comparing max
; subtract element from max
; if result positive, element is new max
CHECK_MAX    LDR R2, R0, #0
             NOT R2, R2
             ADD R2, R2, #1
             ADD R4, R3, R2
             BRp R3_MAX
             LDR R3, R0, #0
R3_MAX
             ADD R0, R0, #1
             ADD R1, R1, #-1
             BRnp CHECK_MAX
             ADD R0, R3, #0

; restore registers used, R4, R3, R2
    LDR R4, R6, #0
    ADD R6, R6, #1
    LDR R3, R6, #0
    ADD R6, R6, #1
    LDR R2, R6, #0
    ADD R6, R6, #1

```

```
        RTI          ; end program

        .END

; section three – test or example code
; When the example runs, the value in the R0 register will be
; maximum value of the array VALUES
        .ORIG x3000
        LEA R0, VALUES
        LD  R1, SIZE
        TRAP x26
        HALT
VALUES  .FILL  #-3
        .FILL  #7
        .FILL  #0
        .FILL  #5
        .FILL  #-1
        .FILL  #4
SIZE    .FILL  #6
        .END
```